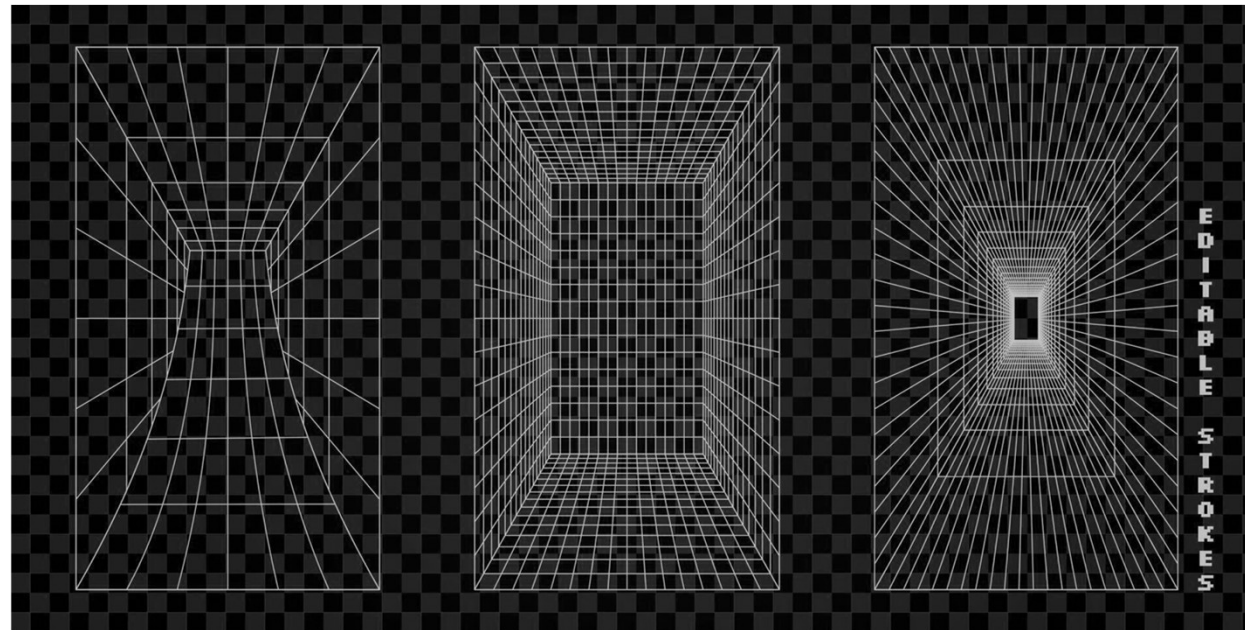




Perspective & LIGHTING

Mastering the Geometry and Physics of Computer Graphics



Lecturer: Giáp Văn Tiến, Ph.D. tiengv@vnu.edu.vn +84865276899
Faculty of engineering and technology

Final project specification: The Interactive Observer

Overall: create an interactive graphics, that simulates a room with light source inside, the wall has picture on it. One ball moved as user pressing keyboard and first-person view changes direction by mouse. The view can be from a CCTV at the corner of the room, which can change the direction.

Each group has max 3 members

Team Roles & Responsibilities

- Student A (The Architect): Geometry, UV Mapping, and Scene Graph.
- Student B (The Engineer): Transformation Matrices, Input Handling, and Camera Logic.
- Student C (The Visual Scientist): GLSL Shaders, Lighting Math, and Material Properties.

⇒ Could be done using OpenGL or Blender

⇒ Use as many AI tools as you wish



Final project Rubric

Criteria	Share	Quality level			
		very good	good	pass	failed
		10 – 8.5	8.4 – 7.0	6.9 – 5.0	4.9 – 0.0
Create object	20%	draw object with 90-100% correct dimension	draw object with 60%-80% correct dimension	draw object with <50% correct dimension	can not draw the object
Mathematical Control	20%	object move exact the same with required types, path and velocity	object move with required types of motions	object moves with time	cannot make object move with time
Visual Realism	20%	make perspective with >80% required factors	make perspective with 60%-80% required factors	make perspective with <60% required factors	cannot make perspective view
Integration & Logic	10%	The code is organized; variables are named clearly; modules talk to each other.	The code is organized; variables are named clearly	The code is organized	Code is a mess
Answer question	30%	Answer > 3/4 of all questions	$\frac{3}{4}$ > Answer > 2/3 of all questions	$\frac{2}{3}$ > Answer > 1/3 of all questions	Answer < 1/3 of all questions

Technical requirements

Module 1: Geometry & Environment (Student A)

The Room: Construct a 3D room based on the provided floor plan. Walls must be distinct entities.

The Actor: Create a high-fidelity sphere (the "Head").

Texture Mapping: One wall must have an image "glued" to it. This requires proper UV Coordinate implementation to prevent stretching.

Asset Loading: (Optional/Bonus) Load the light bulb as a separate .obj or .stl file rather than a primitive.

Module 2: Navigation & Viewports (Student B)

Coordinate Spaces: Manage the transitions between Model Space → World Space → View Space → Projection Space.

The "Head" Controller: WASD/Arrows: Move the ball relative to its "forward" vector.

Mouse Look: Use x and y offsets to rotate the head. Implement a "LookAt" vector logic.

Dual-Camera Toggle: Implement a system to switch the ViewMatrix between:

CCTV Mode: Fixed vec3 position, fixed orientation.

POV Mode: Position and direction updated every frame based on the "Head" coordinates.

Module 3: Illumination & Shading (Student C)

Point Light Logic: The bulb must have a position in World Space.

The Shader: Write a vertex and fragment shader implementing the Phong Reflection Model:

Ambient: Constant base light.

Diffuse: Based on the angle between the surface normal and light direction.

Specular: The "shiny" highlight on the ball that moves with the camera.

Attenuation: Light should naturally get dimmer as the ball moves further away from the bulb.

Final project titles

Group	Theme	Room Layout Variation	Lighting Twist	Camera/Texture Twist
1	The Art Gallery	Long rectangular hallway.	Multiple dim point lights.	Must display 3 different images on 3 different walls.
2	The Night Watch	Square room with central pillar.	Light bulb "flickers" (random intensity).	CCTV view has a "Green Night Vision" shader effect.
3	The Mirror Maze	L-shaped room.	Light is a Spotlight attached to the head.	One wall must be "shiny" (high specular) to act as a mirror.
4	The Disco	Hexagonal room.	Light changes color over time (RGB cycle).	First-person view has a "fisheye" lens distortion.
5	The Laboratory	Two small rooms connected by a door.	One light in each room.	The image on the wall is "Semi-transparent" (Alpha Blending).
6	The Deep Sea	Circular room (Approximated by cylinders).	Light is blue/murky with high attenuation.	The CCTV camera "shakes" slightly as if underwater.
7	The Warehouse	Room filled with 3 static crates (cubes).	Light is placed very low on the floor.	Must use "Mipmapping" for the wall texture to prevent moiré.
8	The Space Station	Room with a "window" (transparent texture).	Light source is "Sunlight" (Directional Light).	The "Head" ball can fly up and down (Z-axis movement).
9	The Haunted House	Narrow corridor with a sharp turn.	Light follows a predefined path (moving light).	Wall texture is a "Portrait" that changes image when clicked.
10	The Neon Grid	Standard Room.	No bulb; the "Ball" itself is the light source.	The floor has a "Grid" texture instead of a solid color.

Geometry Frame, Shading and Lighting

- End-of-class deliverable: orbit camera + cube + Lambert + (Phong/Blinn-Phong) toggle, all in OpenGL C++.
- We will build in 4 checkpoints: P1 baseline → P2 plumbing → P3 Lambert → P4 specular + debug.
- Rule #1: do NOT mix coordinate spaces in dot products ($N \cdot L$, $R \cdot V$, $N \cdot H$).
- Rule #2: normals require a normal matrix when using non-uniform scale.
- Keys: N = debug normals, B = toggle Phong/Blinn-Phong (in final).

Project skeleton (files + dependencies)

```
// Dependencies: GLFW + GLAD + GLM
// Suggested files:
//   main.cpp
//   shaders.glsl (or embedded strings)
// Build (CMake example):
//   find_package(GLFW3 REQUIRED)
//   add_executable(app main.cpp glad.c)
//   target_link_libraries(app glfw opengl32)
// Win

// Include headers
#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
```

Roadmap: Explain ↔ Build ↔ Verify

- Practice 1 (baseline): create window, compile shaders, draw cube, orbit camera.
- Practice 2 (plumbing): output world position & world normal from vertex shader.
- Practice 3 (Lambert): ambient + diffuse (Lambert) in fragment shader.
- Practice 4 (specular): add Phong & Blinn-Phong specular + toggles + debug checks.
- Each checkpoint has a visual test: if it fails, stop and debug before proceeding.

Core pipeline reminder

```
// Vertex shader produces gl_Position
// Fragment shader outputs color

// Today (world-space lighting):
worldPos      = M * vec4(pos,1)
worldNormal = normalMat * normal
L = normalize(lightPos - worldPos)
V = normalize(viewPos  - worldPos)
```

Frames of Reference: pick ONE lighting space

- Object space: mesh authoring coordinates; World space: shared scene coordinates.
- View space: camera at origin; Clip/NDC: after projection.
- We will compute lighting in WORLD space (N, L, V all in world).
- Most lighting bugs: normal in one space, light/view in another.
- Implementation strategy: explicitly name variables worldPos/worldNormal.

Uniforms (world-space lighting contract)

```
// C++ side will upload:
uniform mat4 uM, uV, uP;
uniform mat3 uNormalMat;
uniform vec3 uLightPos;    // world
uniform vec3 uViewPos;     // world

// Fragment shader expects vWorldPos &
vWorldNormal
```


Normals: why you need a normal matrix (remind)

- Normals must remain perpendicular to the surface after transforms.
- Non-uniform scale breaks normals if you use `mat3(M)` directly.
- Correct: **`normalMat = transpose(inverse(mat3(M)))`**.
- Always **normalize**: before and after interpolation.
- We will force a non-uniform scale to validate correctness.

C++: compute normal matrix + forced non-uniform scale

```
glm::mat4 M(1.0f);  
M = glm::scale(M, glm::vec3(1.0f, 0.7f, 1.3f));  
M = glm::rotate(M, t * glm::radians(25.0f),  
glm::vec3(0,1,0));  
  
glm::mat3 normalMat =  
glm::transpose(glm::inverse(glm::mat3(M)));  
// upload:  
glUniformMatrix3fv(uNormalMat, 1, GL_FALSE, value_ptr(normalMat));
```

Practice 1: Create window + GL context (P1_baseline)

- Goal: show a window, initialise GLAD, enable depth testing.
- If you see a blank window: you are still successful at this step.
- Common failures: wrong OpenGL version hints, GLAD not loaded.
- Checkpoint P1.1: `glClearColor()` changes the background.

GLFW + GLAD initialisation

```
if (!glfwInit()) return -1;

// Core profile OpenGL 3.3
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE,
GLFW_OPENGL_CORE_PROFILE);

GLFWwindow* win = glfwCreateWindow(800, 800,
"P1", nullptr, nullptr);
if (!win) { glfwTerminate(); return -1; }
glfwMakeContextCurrent(win);

if
(!gladLoadGLLoader((GLADloadproc) glfwGetProcAddress))
    return -1;

glEnable(GL_DEPTH_TEST);
```

Practice 1: Compile & link shaders (minimal)

- Goal: render pipeline works: vertex shader + fragment shader.
- We start with constant color (no lighting).
- If compile fails: print the shader log and fix it before continuing.
- Checkpoint P1.2: you can call `glUseProgram(prog)` without errors.

Minimal shaders (constant color)

```
// VS (GLSL)
#version 330 core
layout(location=0) in vec3 aPos;
uniform mat4 uM,uV,uP;
void main(){
    gl_Position = uP * uV * uM *
vec4(aPos,1.0);
}

// FS (GLSL)
#version 330 core
out vec4 FragColor;
uniform vec3 uColor;
void main(){ FragColor = vec4(uColor,1.0); }
```

Practice 1: Cube VAO/VBO (positions + normals)

- Goal: create a cube with 36 vertices (12 triangles).
- Even though normals are unused in P1, keep them for P2/P3/P4.
- Attribute layout: location 0 = position, location 1 = normal.
- Checkpoint P1.3: `glDrawArrays(GL_TRIANGLES,0,36)` renders geometry.

VAO layout (stride = 6 floats) function

```
glGenVertexArrays(1, &vao);
glGenBuffers(1, &vbo);

glBindVertexArray(vao);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices),
vertices, GL_STATIC_DRAW);

// pos
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// normal
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)(3*sizeof(float)));
glEnableVertexAttribArray(1);

glBindVertexArray(0);
```

Practice 1: Callback functions (mouse drag + scroll)

- Goal: rotate around the target and zoom in/out smoothly.
- Use yaw/pitch + radius; clamp pitch to avoid flipping.
- Mouse drag updates yaw/pitch; scroll updates radius.
- Checkpoint P1.4: camera movement changes the view of the cube.

Orbit camera (GLM lookAt)

```
static void
framebuffer_size_callback(GLFWwindow*, int w,
int h)
static void mouse_button_callback(GLFWwindow*,
int button, int action, int /*mods*/)
static void cursor_pos_callback(GLFWwindow*
window, double x, double y)
static void scroll_callback(GLFWwindow*, double
/*xoff*/, double yoff)
static void processInput(GLFWwindow* win, float
dt)
```

Practice 1: Orbit camera controls (mouse drag + scroll)

- Goal: rotate around the target and zoom in/out smoothly.
- Use yaw/pitch + radius; clamp pitch to avoid flipping.
- Mouse drag updates yaw/pitch; scroll updates radius.
- Checkpoint P1.4: camera movement changes the view of the cube.

Orbit camera (GLM lookAt)

```
struct OrbitCam {
    float yaw=45, pitch=20, radius=5;
    glm::vec3 target{0,0,0};
    glm::mat4 view() const {
        float cy=cos(radians(yaw)),
        sy=sin(radians(yaw));
        float cp=cos(radians(pitch)),
        sp=sin(radians(pitch));
        glm::vec3 pos = target +
        glm::vec3(radius*cp*cy, radius*sp, radius*cp*sy);
        return glm::lookAt(pos, target, {0,1,0});
    }
};

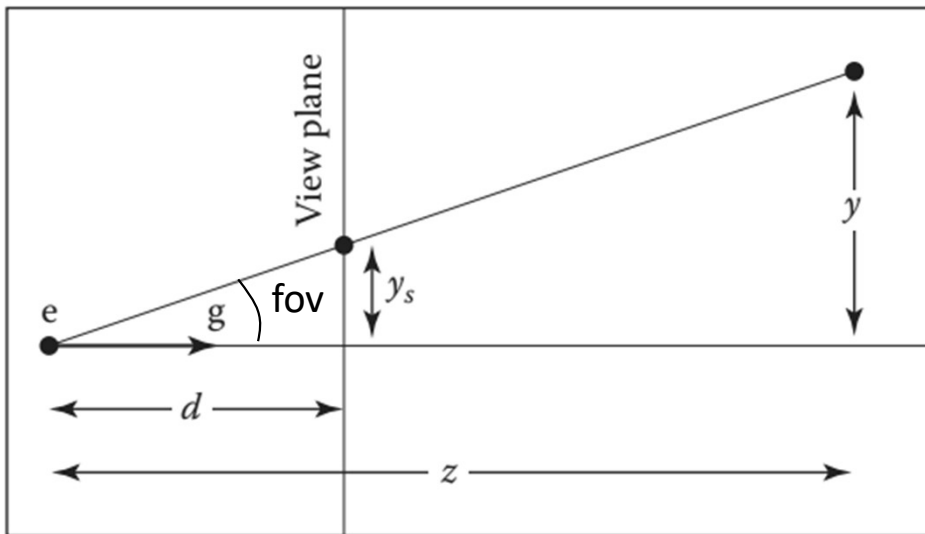
// In callbacks: update yaw/pitch/radius; clamp
pitch [-89,89]
```

Practice 1: Render loop (MVP + draw)

- Goal: the cube rotates and renders with depth testing.
- Compute M, V, P every frame and upload uniforms.
- If cube is clipped: increase camera radius or adjust near/far.
- Checkpoint: rotating cube visible; background clears each frame.

$$y_s = \frac{d}{z} y$$

The Further
the smaller



Render loop core

```
while(!glfwWindowShouldClose(win)){
    float t = (float)glfwGetTime();
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glm::mat4 M(1.0f);
    M = glm::rotate(M, t*glm::radians(30.0f),
    glm::vec3(0,1,0));
    glm::mat4 V = cam.view();
    glm::mat4 P = glm::perspective(glm::radians(60.0f),
    aspect, 0.1f, 100.0f);

    glUseProgram(prog);
    glUniformMatrix4fv(uM,1,GL_FALSE,value_ptr(M));
    glUniformMatrix4fv(uV,1,GL_FALSE,value_ptr(V));
    glUniformMatrix4fv(uP,1,GL_FALSE,value_ptr(P));
    glUniform3f(uColor, 0.9f, 0.6f, 0.3f);

    glBindVertexArray(cubeVAO);
    glDrawArrays(GL_TRIANGLES,0,36);
    glfwSwapBuffers(win);
    glfwPollEvents();
}
```

`perspective(fov,view_ratio,d,z)`

Practice 2: Add worldPos + worldNormal outputs

- Goal: pass correct world-space data to the fragment shader.
- Vertex shader must output vWorldPos and vWorldNormal.
- Normal transform uses uNormalMat (transpose-inverse of mat3(M)).
- Checkpoint: a “debug normals” fragment shader shows stable colors.

Note: never upload the data which is not declared in the GLSL (runtime error)

Vertex shader (GLSL world outputs)

```
#version 330 core
layout(location=0) in vec3 aPos;
layout(location=1) in vec3 aNormal;

uniform mat4 uM,uV,uP;
uniform mat3 uNormalMat;

out vec3 vWorldPos;
out vec3 vWorldNormal;

void main(){
    vec4 wp = uM * vec4(aPos,1.0);
    vWorldPos = wp.xyz;
    vWorldNormal = normalize(uNormalMat * aNormal);
    gl_Position = uP * uV * wp;
}
```


Practice 2: Debug normals visualization (key N later)

- Goal: confirm normals are correct before writing lighting.
- Technique: map normal from $[-1,1]$ to $[0,1]$ as color.
- Expected: face colors rotate with the object; camera movement should not change colors.
- If colors distort under non-uniform scale: your normal matrix is wrong.

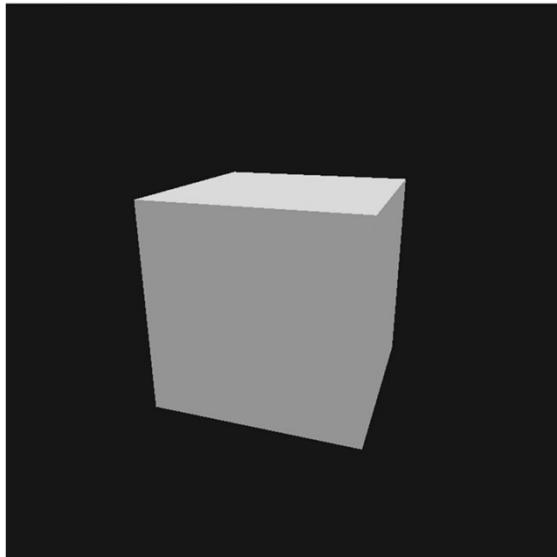
Fragment shader (debug normals)

```
#version 330 core
in vec3 vWorldNormal;
out vec4 FragColor;

void main(){
    vec3 N = normalize(vWorldNormal);
    FragColor = vec4(N*0.5 + 0.5, 1.0);
}
```

Practice 2: C++ uploads (uNormalMat) + forced test

- Compute normalMat each frame from M and upload to shader.
- Force a non-uniform scale so incorrect normal handling is obvious.
- Checkpoint: debug-normal colors remain “clean” (no unexpected gradients).
- If needed: temporarily disable rotation to inspect each face.



C++: upload normalMat

```
glm::mat4 M(1.0f);
M = glm::scale(M, glm::vec3(1.0f, 0.7f, 1.3f));
M = glm::rotate(M, t*glm::radians(25.0f),
glm::vec3(0,1,0));

glm::mat3 normalMat =
glm::transpose(glm::inverse(glm::mat3(M)));

// Uniform locations:
Glint uNormalMat =
glGetUniformLocation(prog,"uNormalMat");
glUniformMatrix3fv(uNormalMat,1,GL_FALSE,glm::value_ptr(normalMat));
```

Practice 3: Lambert diffuse + ambient (P3_lambert)

- Goal: implement ambient + Lambert diffuse (no specular yet).
- Lambert: $\text{ndotl} = \max(\text{dot}(\mathbf{N}, \mathbf{L}), 0)$.
- Ambient: $K_a * \text{ambientColor}$ (simple indirect-light approximation).
- Checkpoint: faces brighten/darken as light moves around.

Fragment shader (Lambert + ambient)

```
#version 330 core
in vec3 vWorldPos;
in vec3 vWorldNormal;
out vec4 FragColor;

uniform vec3 uLightPos;
uniform vec3 uLightColor;
uniform vec3 uAmbientColor;
uniform vec3 uKa;
uniform vec3 uKd;

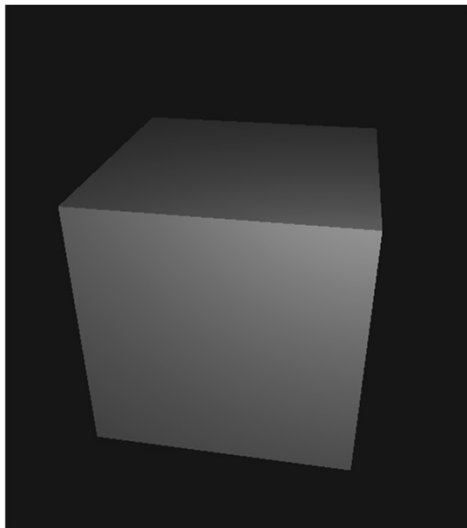
void main(){
    vec3 N = normalize(vWorldNormal);
    vec3 L = normalize(uLightPos - vWorldPos);

    vec3 ambient = uKa * uAmbientColor;
    float ndotl = max(dot(N,L), 0.0);
    vec3 diffuse = uKd * ndotl * uLightColor;

    FragColor = vec4(ambient + diffuse, 1.0);
}
```

Practice 3: C++ uniforms for light + material

- Upload world-space light position and colors.
- Upload material Ka and Kd (ambient and diffuse reflectance).
- Start with fixed light; then animate it on a circle.
- Checkpoint: light motion changes the brightest face over time.



C++: set light/material uniforms

```
GLint uLightPos = glGetUniformLocation(prog, "uLightPos");
GLint uLightColor = glGetUniformLocation(prog, "uLightColor");
GLint uAmbientColor = glGetUniformLocation(prog, "uAmbientColor");
GLint uKa = glGetUniformLocation(prog, "uKa");
GLint uKd = glGetUniformLocation(prog, "uKd");

glm::vec3 lightPos(2.5f, 2.0f, 2.5f);
glm::vec3 lightColor(1.0f);
glm::vec3 ambientColor(0.2f);

glm::vec3 Ka(0.15f);
glm::vec3 Kd(0.9f, 0.6f, 0.3f);
//render loop
// animate (optional):
// lightPos = {2.5f*cos(t), 1.8f, 2.5f*sin(t)};

glUniform3fv(uLightPos,1,value_ptr(lightPos));
glUniform3fv(uLightColor,1,value_ptr(lightColor));
glUniform3fv(uAmbientColor,1,value_ptr(ambientColor));
glUniform3fv(uKa,1,value_ptr(Ka));
glUniform3fv(uKd,1,value_ptr(Kd));
```

Practice 4: Add view vector + specular uniforms (P4_final)

- Goal: add specular highlight that responds to camera movement.
- Compute $V = \text{normalize}(\text{viewPos} - \text{worldPos})$.
- Add material K_s and shininess exponent.
- Checkpoint: highlight moves when you orbit the camera.

Add camera position function to camera struct

```
glm::vec3 position() const {          float cy =
    std::cos(glm::radians(yaw));      float sy =
    std::sin(glm::radians(yaw));      float cp =
    std::cos(glm::radians(pitch));    float sp =
    std::sin(glm::radians(pitch));    return
    target + glm::vec3(radius * cp * cy, radius * sp, radius *
    cp * sy);
}
```

C++: compute and upload viewPos + specular params

```
GLint uViewPos = glGetUniformLocation(prog,
    "uViewPos");
GLint uShininess = glGetUniformLocation(prog,
    "uShininess");
GLint uSh = glGetUniformLocation(prog, "uSh");
GLint uKs = glGetUniformLocation(prog, "uKs");
//render loop
glm::vec3 viewPos = cam.position();

glm::vec3 Ks(0.7f);
float shininess = 64.0f;

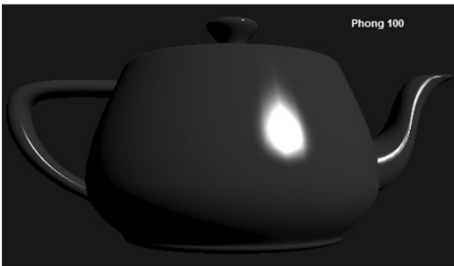
glUniform3fv(uViewPos,1,value_ptr(viewPos));
glUniform3fv(uKs,1,value_ptr(Ks));
glUniform1f(uShininess, shininess);

//FS GLSL
uniform vec3 uViewPos;
uniform vec3 uKs;    // specular reflectance
uniform float uShininess;
```

Practice 4: Specular — Phong and Blinn-Phong (toggle B)

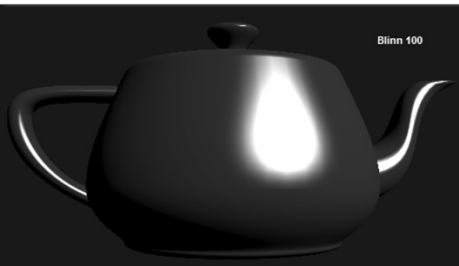
- Phong uses reflection vector R; Blinn-Phong uses half-vector H.
- We keep Lambert diffuse and add specular on top.
- Specular only when $\text{ndotl} > 0$ (light on the front side).
- Toggle with uniform `uUseBlinn` (set by key B).

Phong



slower, Highlights are often perfectly round and intense. Can fail (cut off) if the light/view angle is $>90^\circ$

Blinn Phong ✓



Faster, highlights are softer and slightly more "stretched" or elliptical, which often looks more realistic

Phong vs Blinn-Phong specular

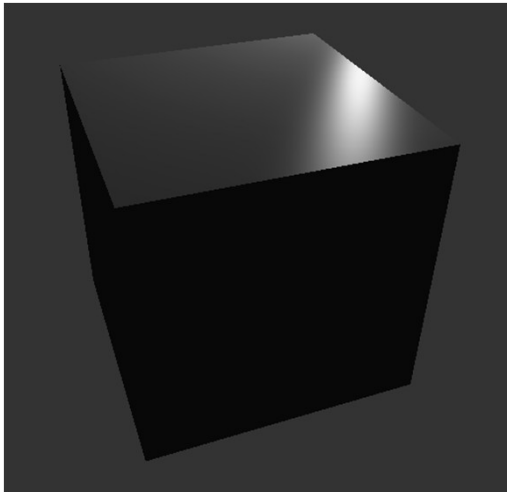
```
//C++
GLint uUseBlinn = glGetUniformLocation(prog,
"uUseBlinn");

//Fragment shader GLSL
uniform bool uUseBlinn;

vec3 V = normalize(uViewPos - vWorldPos);
float spec = 0.0;
if (ndotl > 0.0) {
    if (uUseBlinn) {
        vec3 H = normalize(L + V);
        spec = pow(max(dot(N, H), 0.0), uShininess);
    } else {
        vec3 R = reflect(-L, N);
        spec = pow(max(dot(R, V), 0.0), uShininess);
    }
}
vec3 specular = uKs * spec * uLightColor;
FragColor = vec4(ambient + diffuse + specular,
1.0);
```

Practice 4: Toggle keys + debug normals (N)

- Implement edge-trigger toggles so keys do not flip continuously.
- B toggles `uUseBlinn`; N toggles `uDebugNormals`.
- When debugging, first view normals; then check vector normalization.
- Checkpoint: toggles work reliably (one press = one toggle).



Blinn
or
Phong?

C++: edge-trigger toggles

```
static bool gUseBlinn=true, debugN=false;
static bool lastB=false, lastN=false;
GLint uDebugNormals = glGetUniformLocation(prog,
"uDebugNormals");
//inside processInput()
bool b = glfwGetKey(win, GLFW_KEY_B)==GLFW_PRESS;
bool n = glfwGetKey(win, GLFW_KEY_N)==GLFW_PRESS;
if (b && !lastB) gUseBlinn = !gUseBlinn;
if (n && !lastN) gDebugNormals = ! gDebugNormals;
lastB=b; lastN=n;

//render loop
// uniforms:
glUniform1i(uUseBlinn, gUseBlinn?1:0);
glUniform1i(uDebugNormals, gDebugNormals?1:0);

//FS GLSL
// Debug normals visualization
if (uDebugNormals) {
    FragColor = vec4(N * 0.5 + 0.5, 1.0);
    return;
}
```

Final validation checklist (end-of-class demo)

- Orbit camera works (mouse drag + scroll).
- Lambert shading changes correctly when light moves.
- Specular highlight moves with camera (proves V is correct).
- Normals debug mode looks stable under non-uniform scale (proves normalMat).
- Phong vs Blinn-Phong toggle shows visibly different highlights.

Common bug patterns (quick fixes)

```
// 1) Lighting "stuck" to camera: mixing spaces
// Fix: ensure
worldPos/worldNormal/lightPos/viewPos are all
world-space

// 2) Specular missing: ndotl <= 0 or vectors
not normalized
// Fix: normalize N,L,V (and H)

// 3) Wrong under non-uniform scale
// Fix: normalMat =
transpose(inverse(mat3(M)))
```


Homework extension (small, aligned with today)

- Add a second object with different Kd/Ks/shininess.
- Add a key to freeze/unfreeze light animation.
- Submit 3 screenshots: Lambert-only, Phong, Blinn-Phong.
- Write a short note: which space you used, why normal matrix is needed, and your debugging strategy.

Material presets (examples)

```
// Plastic
Ka=vec3(0.10); Kd=vec3(0.9,0.2,0.2);
Ks=vec3(0.6); shin=64;
```

```
// Matte
Ka=vec3(0.15); Kd=vec3(0.6,0.6,0.6);
Ks=vec3(0.05); shin=8;
```

```
// Shiny
Ka=vec3(0.05); Kd=vec3(0.2,0.2,0.2);
Ks=vec3(1.0); shin=256;
```